In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

sns.set_theme(style="whitegrid")
```

> **Observation:** These lines load the tools we need: Pandas for data tables, NumPy for numbers, and Matplotlib and Seaborn for charts. Nothing is shown because this step only gets things ready.

1. Load the Dataset

In [2]:
```
df = pd.read_csv("student-mat.csv", sep=";")

df.head()
```

Out[2]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reason
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	course
1	GP	F	17	U	GT3	T	1	1	at_home	other	course
2	GP	F	15	U	LE3	T	1	1	at_home	other	other
3	GP	F	15	U	GT3	T	4	2	health	services	home
4	GP	F	16	U	GT3	T	3	3	other	other	home

> **Observation:** The data file has been loaded. Each row is one student and each column is one detail about them (school, sex, age, grades and so on). The first five students are all girls from school GP.

In [3]:
```
df.tail()
```

Out[3]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reason
390	MS	M	20	U	LE3	A	2	2	services	services	course
391	MS	M	17	U	LE3	T	3	1	services	services	course
392	MS	M	21	R	GT3	T	1	1	other	other	course
393	MS	M	18	R	LE3	T	3	2	services	other	course
394	MS	M	19	U	LE3	T	1	1	other	at_home	course

> **Observation:** These are the last five students in the data. They are from school MS, which tells us the file contains students from two different schools.

In [4]:
```
print("Shape:", df.shape)
```

Out[4]:
```
Shape: (395, 33)
```

> **Observation:** The data has 395 students (rows) and 33 details about each student (columns).

In [5]:
```
df.info()
```

Out[5]:
```
<class 'pandas.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    str
 1   sex         395 non-null    str
 2   age         395 non-null    int64
 3   address     395 non-null    str
 4   famsize     395 non-null    str
 5   Pstatus     395 non-null    str
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    str
 9   Fjob        395 non-null    str
 10  reason      395 non-null    str
 11  guardian    395 non-null    str
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    str
 16  famsup      395 non-null    str
 17  paid        395 non-null    str
 18  activities  395 non-null    str
 19  nursery     395 non-null    str
 20  higher      395 non-null    str
 21  internet    395 non-null    str
 22  romantic    395 non-null    str
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), str(17)
memory usage: 102.0 KB
```

> **Observation:** All 33 columns have 395 values, so nothing is missing. 16 columns are numbers and 17 are text, such as school, sex and job.

In [6]:
```
df.describe()
```

Out[6]:

	age	Medu	Fedu	traveltime	studytime	failures	famrel
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000

	age	Medu	Fedu	traveltime	studytime	failures	famrel
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944304
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896659
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000

> **Observation:** On average students are about 17 years old and the average final grade (G3) is about 10.4 out of 20. Grades go from 0 to 20. Absences go from 0 to 75, but the typical student has only about 4.

2. Selecting and Indexing Data

In [7]:
```
df["G3"].head()
```

Out[7]:
```
0     6
1     6
2    10
3    15
4    10
Name: G3, dtype: int64
```

> **Observation:** This shows only the final grades of the first five students: 6, 6, 10, 15 and 10.

In [8]:
```
df[["sex", "age", "studytime", "absences", "G1", "G2", "G3"]].head()
```

Out[8]:

	sex	age	studytime	absences	G1	G2	G3
0	F	18	2	6	5	6	6
1	F	17	2	4	5	5	6
2	F	15	2	10	7	8	10
3	F	15	3	2	15	14	15
4	F	16	2	4	6	10	10

> **Observation:** Now we see only the columns we picked. Student 3 studied more (level 3) and had only 2 absences, and got a high grade of 15. The first two students have low grades of 6.

In [9]:
```
df.iloc[0:5, 0:8]
```

Out[9]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu
0	GP	F	18	U	GT3	A	4	4
1	GP	F	17	U	GT3	T	1	1

	school	sex	age	address	famsize	Pstatus	Medu	Fedu
2	GP	F	15	U	LE3	T	1	1
3	GP	F	15	U	GT3	T	4	2
4	GP	F	16	U	GT3	T	3	3

> **Observation:** This picks the first 5 rows and first 8 columns by position. It gives a small slice of the data without the grades.

3. Filtering Data

In [10]:
```
high_score_students = df[df["G3"] > 15]
high_score_students.head()
```

Out[10]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reas
8	GP	M	15	U	LE3	A	3	2	services	other	hor
14	GP	M	15	U	GT3	A	2	2	other	other	hor
22	GP	M	16	U	LE3	T	4	2	teacher	other	cour
31	GP	M	15	U	GT3	T	4	4	services	services	reputati
32	GP	M	15	R	GT3	T	4	3	teacher	at_home	cour

> **Observation:** 40 students scored above 15 in the final grade. Most of the ones shown have 0 or 2 absences.

In [11]:
```
low_absence_students = df[df["absences"] < 5]
low_absence_students.head()
```

Out[11]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reason
1	GP	F	17	U	GT3	T	1	1	at_home	other	course
3	GP	F	15	U	GT3	T	4	2	health	services	home
4	GP	F	16	U	GT3	T	3	3	other	other	home
6	GP	M	16	U	LE3	T	2	2	other	other	home
8	GP	M	15	U	LE3	A	3	2	services	other	home

> **Observation:** 244 of the 395 students missed fewer than 5 classes, so most students attend regularly.

In [12]:
```
filtered_students = df[(df["studytime"] > 2) & (df["G3"] >= 15)]
filtered_students.head()
```

Out[12]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reaso
3	GP	F	15	U	GT3	T	4	2	health	services	hom

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	reaso
14	GP	M	15	U	GT3	A	2	2	other	other	hom
36	GP	M	15	U	LE3	T	4	3	teacher	services	hom
37	GP	M	16	R	GT3	A	4	4	other	teacher	reputatio
47	GP	M	16	U	GT3	T	4	3	health	services	reputatio

> **Observation:** Only 24 students studied more than 2 hours and also scored 15 or more. Studying a lot together with a high score is not very common.

4. Creating New Columns

In [13]:
```
df["Average_Grade"] = df[["G1", "G2", "G3"]].mean(axis=1)

df[["G1", "G2", "G3", "Average_Grade"]].head()
```

Out[13]:

	G1	G2	G3	Average_Grade
0	5	6	6	5.666667
1	5	5	6	5.333333
2	7	8	10	8.333333
3	15	14	15	14.666667
4	6	10	10	8.666667

> **Observation:** A new column called Average_Grade was added. For example, the first student's three grades (5, 6, 6) give an average of about 5.67.

In [14]:
```
df["Grade_Improvement"] = df["G3"] - df["G1"]

df[["G1", "G3", "Grade_Improvement"]].head()
```

Out[14]:

	G1	G3	Grade_Improvement
0	5	6	1
1	5	6	1
2	7	10	3
3	15	15	0
4	6	10	4

> **Observation:** A new column shows how much the grade changed from the first test (G1) to the final (G3). The first five students stayed the same or improved. Across all students, more students (159) went down than went up (143).

In [15]:
```
def performance_category(grade):
    if grade >= 15:
        return "Excellent"
```

```
    elif grade >= 10:
        return "Good"
    else:
        return "Needs Improvement"

df["Performance_Category"] = df["G3"].apply(performance_category)

df[["G3", "Performance_Category"]].head(10)
```

Out[15]:

	G3	Performance_Category
0	6	Needs Improvement
1	6	Needs Improvement
2	10	Good
3	15	Excellent
4	10	Good
5	15	Excellent
6	11	Good
7	6	Needs Improvement
8	19	Excellent
9	15	Excellent

> **Observation:** Each student now gets a label based on the final grade: 15 or above is Excellent, 10 to 14 is Good, and below 10 is Needs Improvement. Over the whole data, 192 students are Good, 130 Need Improvement and 73 are Excellent.

5. Missing Values and Data Cleaning

In [16]:
```
df.isnull().sum()
```

Out[16]:
```
school              0
sex                 0
age                 0
address             0
famsize             0
Pstatus             0
Medu                0
Fedu                0
Mjob                0
Fjob                0
reason              0
guardian            0
traveltime          0
studytime           0
failures            0
schoolsup           0
famsup              0
paid                0
activities          0
nursery             0
higher              0
internet            0
romantic            0
```

```
famrel                0
freetime              0
goout                 0
Dalc                  0
Walc                  0
health                0
absences              0
G1                    0
G2                    0
G3                    0
Average_Grade         0
Grade_Improvement     0
Performance_Category  0
dtype: int64
```

> **Observation:** Every column shows 0, which means there are no missing values anywhere in the data.

In [17]:
```
print("Total missing values:", df.isnull().sum().sum())
```

Out[17]:
```
Total missing values: 0
```

> **Observation:** The total number of missing values is 0, so the data is already clean.

In [18]:
```
df_filled = df.copy()
df_filled["G3"] = df_filled["G3"].fillna(df_filled["G3"].mean())

df_filled["G3"].isnull().sum()
```

Out[18]:
```
np.int64(0)
```

> **Observation:** The code would fill missing final grades with the average grade. The result is 0 because there was nothing missing to fill.

In [19]:
```
df_ffill = df.copy()
df_ffill = df_ffill.ffill()

df_bfill = df.copy()
df_bfill = df_bfill.bfill()

print("Forward-fill completed:", df_ffill.isnull().sum().sum(), "missing values rema
print("Backward-fill completed:", df_bfill.isnull().sum().sum(), "missing values re
```

Out[19]:
```
Forward-fill completed: 0 missing values remain
Backward-fill completed: 0 missing values remain
```

> **Observation:** Forward fill and backward fill are two ways of filling empty cells from neighbouring rows. Both leave 0 missing values because the data had none.

In [20]:
```
df_dropna = df.dropna()
```

```
print("Original rows:", len(df))
print("Rows after dropna():", len(df_dropna))
```

Out[20]:
```
Original rows: 395
Rows after dropna(): 395
```

> **Observation:** Dropping rows with missing values removed nothing. We still have all 395 rows.

6. Grouping and Aggregation

In [21]:
```
df.groupby("sex")["G3"].mean()
```

Out[21]:
```
sex
F     9.966346
M    10.914439
Name: G3, dtype: float64
```

> **Observation:** Boys have a higher average final grade (about 10.9) than girls (about 10.0), a difference of nearly 1 mark.

In [22]:
```
df.groupby("school")["G3"].mean()
```

Out[22]:
```
school
GP    10.489971
MS     9.847826
Name: G3, dtype: float64
```

> **Observation:** School GP has a slightly higher average grade (about 10.5) than school MS (about 9.8).

In [23]:
```
df.groupby("studytime")["G3"].mean().sort_values(ascending=False)
```

Out[23]:
```
studytime
3    11.400000
4    11.259259
2    10.171717
1    10.047619
Name: G3, dtype: float64
```

> **Observation:** Students who study 3 hours or 4 hours have the best average grades (about 11.4 and 11.3). Those who study less (levels 1 and 2) average about 10.

In [24]:
```
df.groupby("sex").agg(
    Average_G3=("G3", "mean"),
    Maximum_G3=("G3", "max"),
    Minimum_G3=("G3", "min"),
    Average_Absences=("absences", "mean")
)
```

Out[24]:

	Average_G3	Maximum_G3	Minimum_G3	Average_Absences
sex				
F	9.966346	19	0	6.216346
M	10.914439	20	0	5.144385

> **Observation:** Boys score a little higher on average, and the top grade is 20 for boys and 19 for girls. Both have a lowest grade of 0. Girls have more absences on average (about 6.2) than boys (about 5.1).

7. Sorting

In [25]:
```
df.sort_values(by="G3", ascending=False)[
    ["school", "sex", "studytime", "absences", "G3"]
].head(10)
```

Out[25]:

	school	sex	studytime	absences	G3
47	GP	M	4	4	20
374	MS	F	3	0	19
110	GP	M	1	6	19
113	GP	M	1	10	19
8	GP	M	2	0	19
286	GP	F	3	5	19
36	GP	M	3	2	18
104	GP	M	2	0	18
129	GP	M	1	8	18
107	GP	M	3	2	18

> **Observation:** The best student scored 20 and studied at the highest level. Several others scored 19 with very different study times, so a high grade is possible even with low study time.

In [26]:
```
df.sort_values(by="absences", ascending=False)[
    ["sex", "absences", "G3"]
].head(10)
```

Out[26]:

	sex	absences	G3
276	F	75	9
183	F	56	8
74	F	54	11
315	F	40	11
307	M	38	8
280	M	30	8

	sex	absences	G3
205	F	28	9
103	F	26	6
40	F	25	11
198	F	24	18

> **Observation:** The student with the most absences missed 75 classes, yet still scored 9. One student with 24 absences still got 18. So missing many classes does not always mean a low grade.

8. Pivot Tables

In [27]:
```
pivot_table = pd.pivot_table(
    df,
    values="G3",
    index="sex",
    columns="studytime",
    aggfunc="mean"
)

pivot_table
```

Out[27]:

studytime	1	2	3	4
sex				
F	9.851852	9.495575	10.725490	11.0
M	10.115385	11.070588	13.857143	11.7

> **Observation:** Boys who study at level 3 have the highest average (about 13.9). Girls who study at level 2 have the lowest (about 9.5). Averages generally rise with more study time, but not always.

In [28]:
```
pd.pivot_table(
    df,
    values="G3",
    index="school",
    columns="sex",
    aggfunc="mean"
)
```

Out[28]:

sex	F	M
school		
GP	9.972678	11.060241
MS	9.920000	9.761905

> **Observation:** At school GP boys score higher than girls (about 11.1 vs 10.0). At school MS boys and girls score almost the same (about 9.8 and 9.9).

9. Exporting Data

In [29]:
```
df.to_csv("student_performance_processed.csv", index=False)

print("Processed dataset saved as student_performance_processed.csv")
```

Out[29]:
```
Processed dataset saved as student_performance_processed.csv
```

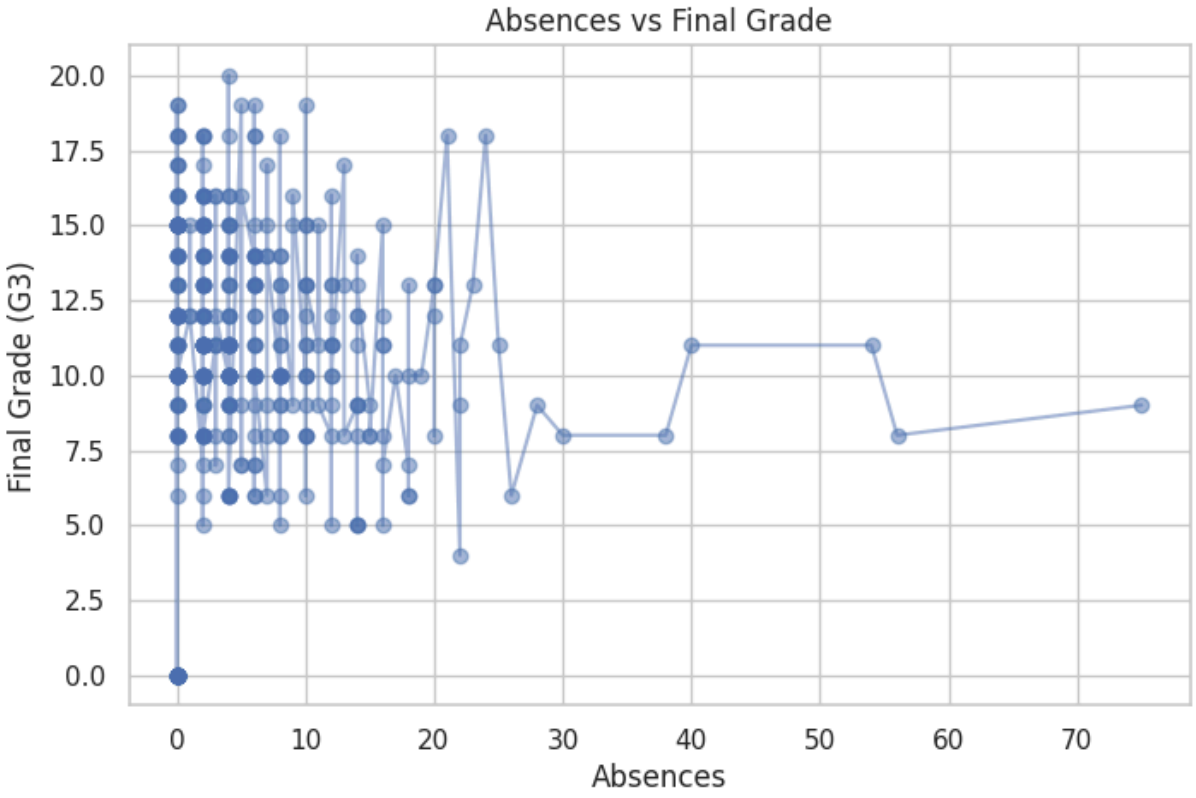
> **Observation:** The cleaned data, with the new columns we added, was saved as a new CSV file.

Matplotlib Visualizations

10. Line Plot — Absences vs Final Grade

In [30]:
```
plt.figure(figsize=(8, 5))
sorted_df = df.sort_values("absences")
plt.plot(sorted_df["absences"], sorted_df["G3"], marker="o", linestyle="-", alpha=0
plt.xlabel("Absences")
plt.ylabel("Final Grade (G3)")
plt.title("Absences vs Final Grade")
plt.show()
```

Out[30]:



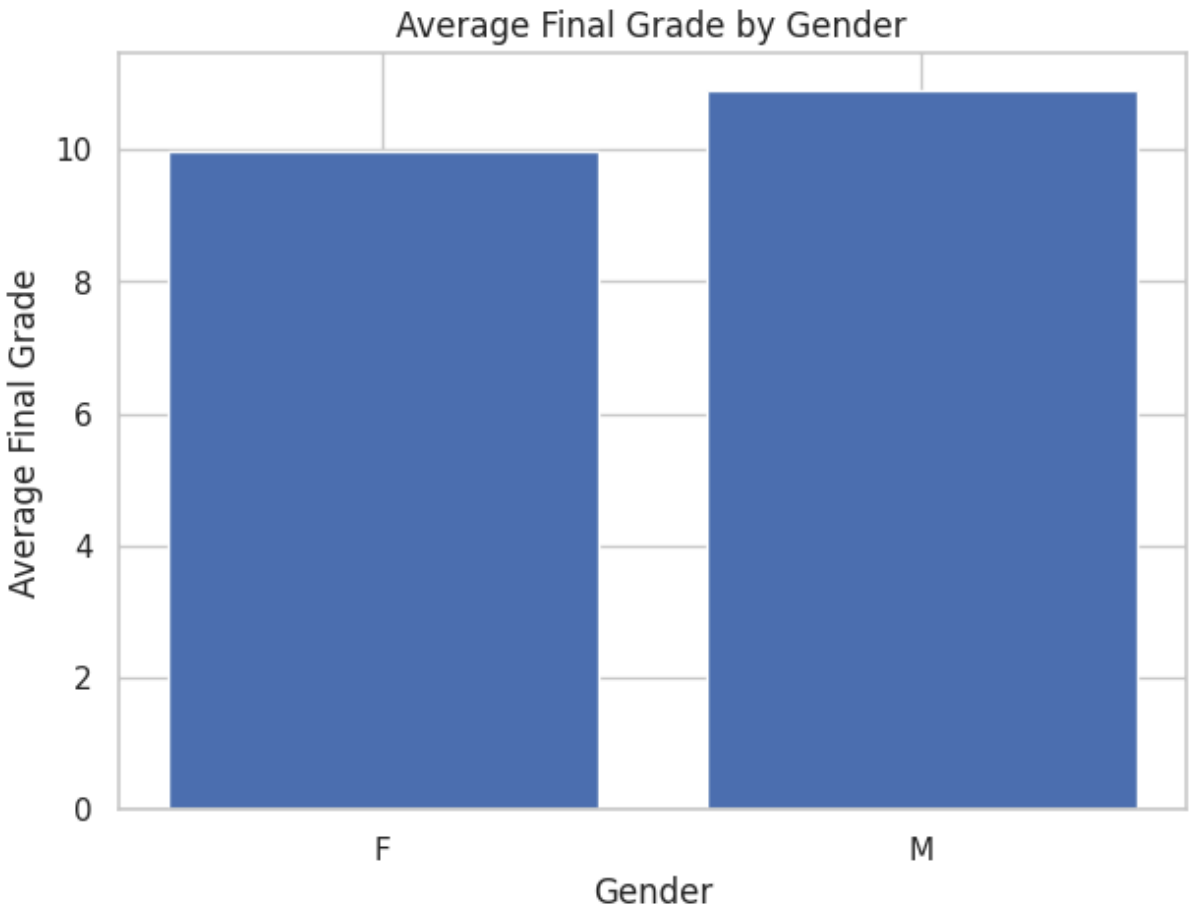
> **Observation:** The line zigzags up and down and does not show a clear trend. Most students have few absences, and their grades range from 0 to 20. The few students with very high absences make the line stretch far to the right.

11. Bar Chart — Average Final Grade by Gender

In [31]:
```
avg_gender_grade = df.groupby("sex")["G3"].mean()

plt.figure(figsize=(7, 5))
plt.bar(avg_gender_grade.index, avg_gender_grade.values)
plt.xlabel("Gender")
plt.ylabel("Average Final Grade")
plt.title("Average Final Grade by Gender")
plt.show()
```

Out[31]:



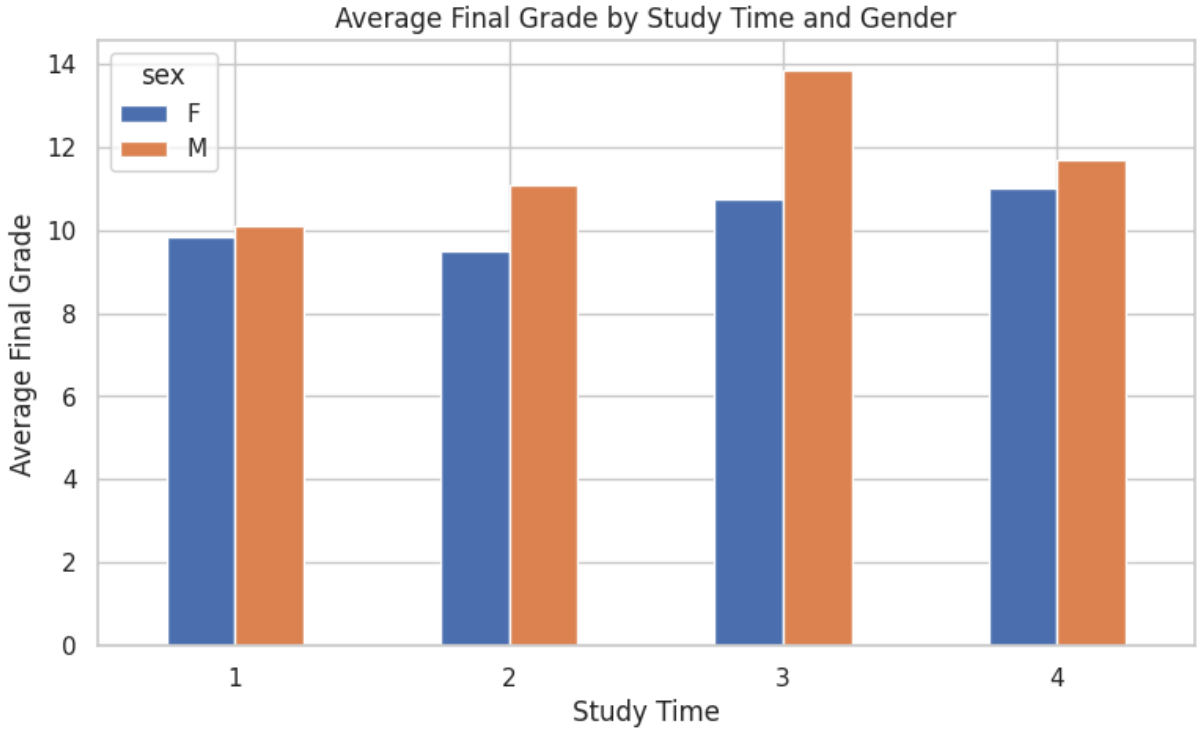
> **Observation:** The boys' bar is slightly taller than the girls' bar. Boys average about 10.9 and girls about 10.0.

12. Grouped Bar Chart — Average Grades by Study Time and Gender

In [32]:
```
grouped = df.groupby(["studytime", "sex"])["G3"].mean().unstack()

ax = grouped.plot(kind="bar", figsize=(9, 5))
ax.set_xlabel("Study Time")
ax.set_ylabel("Average Final Grade")
ax.set_title("Average Final Grade by Study Time and Gender")
plt.xticks(rotation=0)
plt.show()
```

Out[32]:



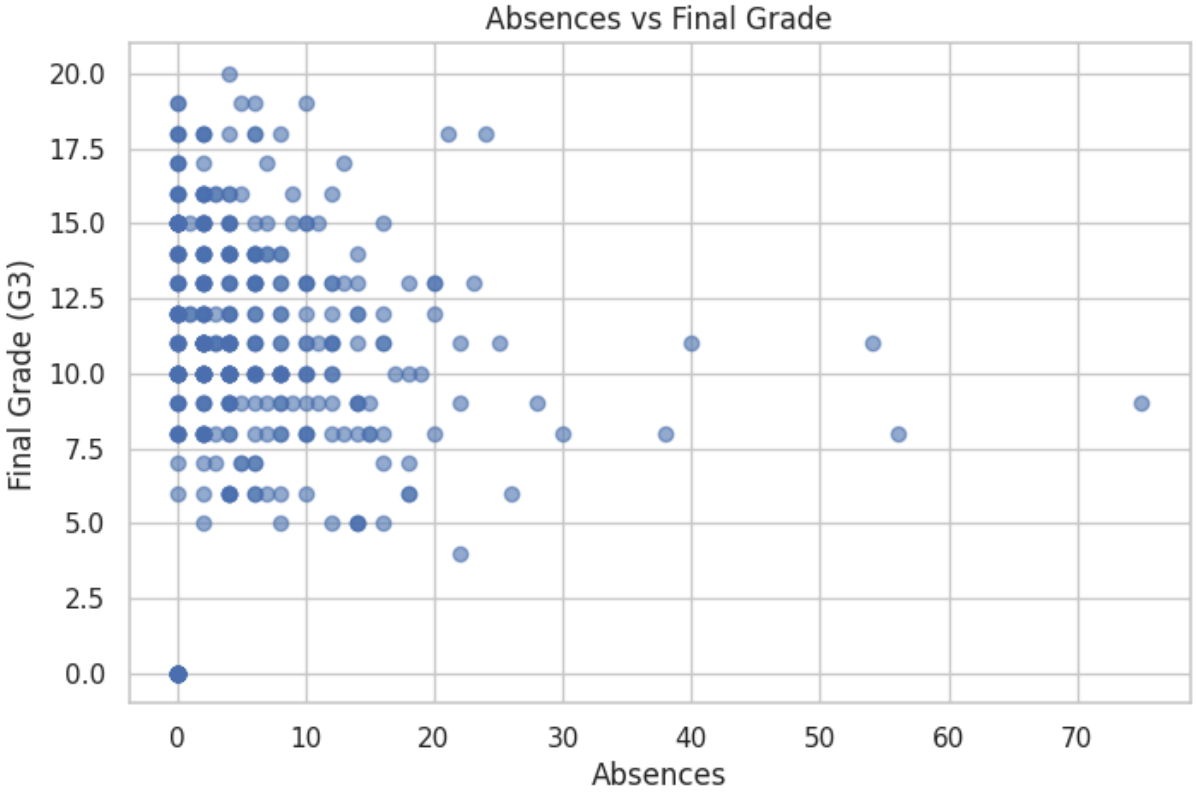
> **Observation:** For boys, the average grade rises up to study level 3 (about 13.9) and then falls a little at level 4. For girls, it dips at level 2 and then rises at levels 3 and 4. More study time mostly helps, but the pattern is not perfectly smooth.

13. Scatter Plot — Absences vs Final Grade

In [33]:

```
plt.figure(figsize=(8, 5))
plt.scatter(df["absences"], df["G3"], alpha=0.6)
plt.xlabel("Absences")
plt.ylabel("Final Grade (G3)")
plt.title("Absences vs Final Grade")
plt.show()
```

Out[33]:



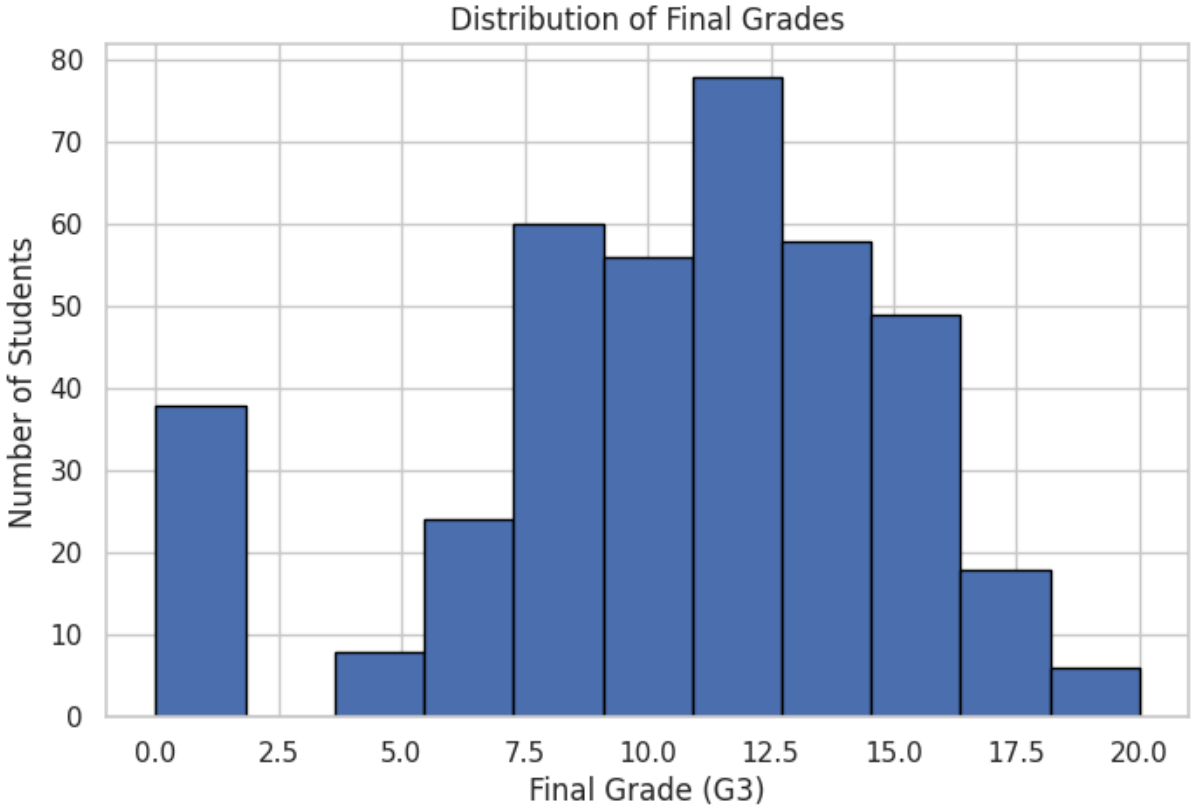
> **Observation:** Most dots are bunched on the left, meaning most students have under 20 absences. There is no clear line or pattern, so absences alone do not predict the final grade. A row of dots at grade 0 is visible.

14. Histogram — Final Grade

In [34]:

```
plt.figure(figsize=(8, 5))
plt.hist(df["G3"], bins=11, edgecolor="black")
plt.xlabel("Final Grade (G3)")
plt.ylabel("Number of Students")
plt.title("Distribution of Final Grades")
plt.show()
```

Out[34]:



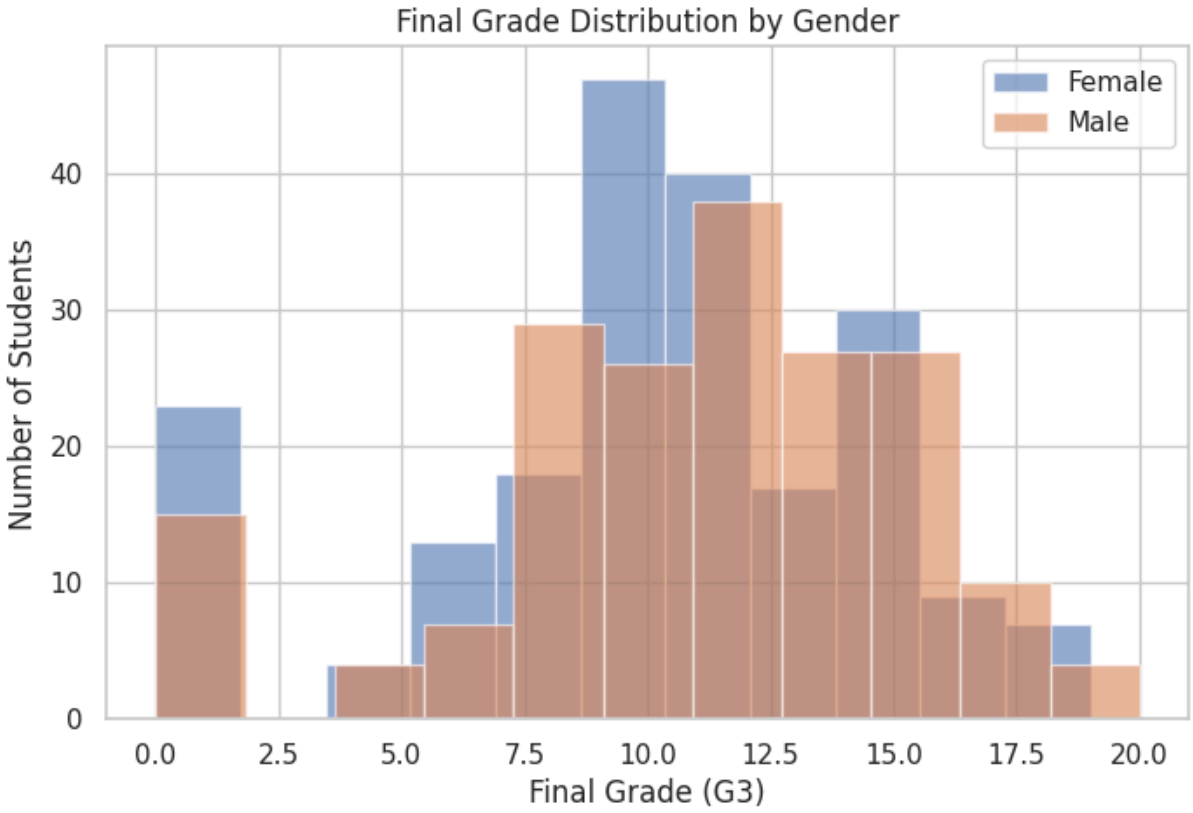
> **Observation:** Most students score between 8 and 14, and the tallest bars are at 10 and 11. There is also a separate bar at 0 (38 students), and very few students score 18 or more.

15. Histogram — Final Grade by Gender

In [35]:

```
plt.figure(figsize=(8, 5))
plt.hist(df[df["sex"] == "F"]["G3"], bins=11, alpha=0.6, label="Female")
plt.hist(df[df["sex"] == "M"]["G3"], bins=11, alpha=0.6, label="Male")
plt.xlabel("Final Grade (G3)")
plt.ylabel("Number of Students")
plt.title("Final Grade Distribution by Gender")
plt.legend()
plt.show()
```

Out[35]:



> **Observation:** Boys and girls have a similar shape. Girls have more students in total (208 vs 187) and more zero scores (23 vs 15). Boys have slightly more students in the higher grade range.

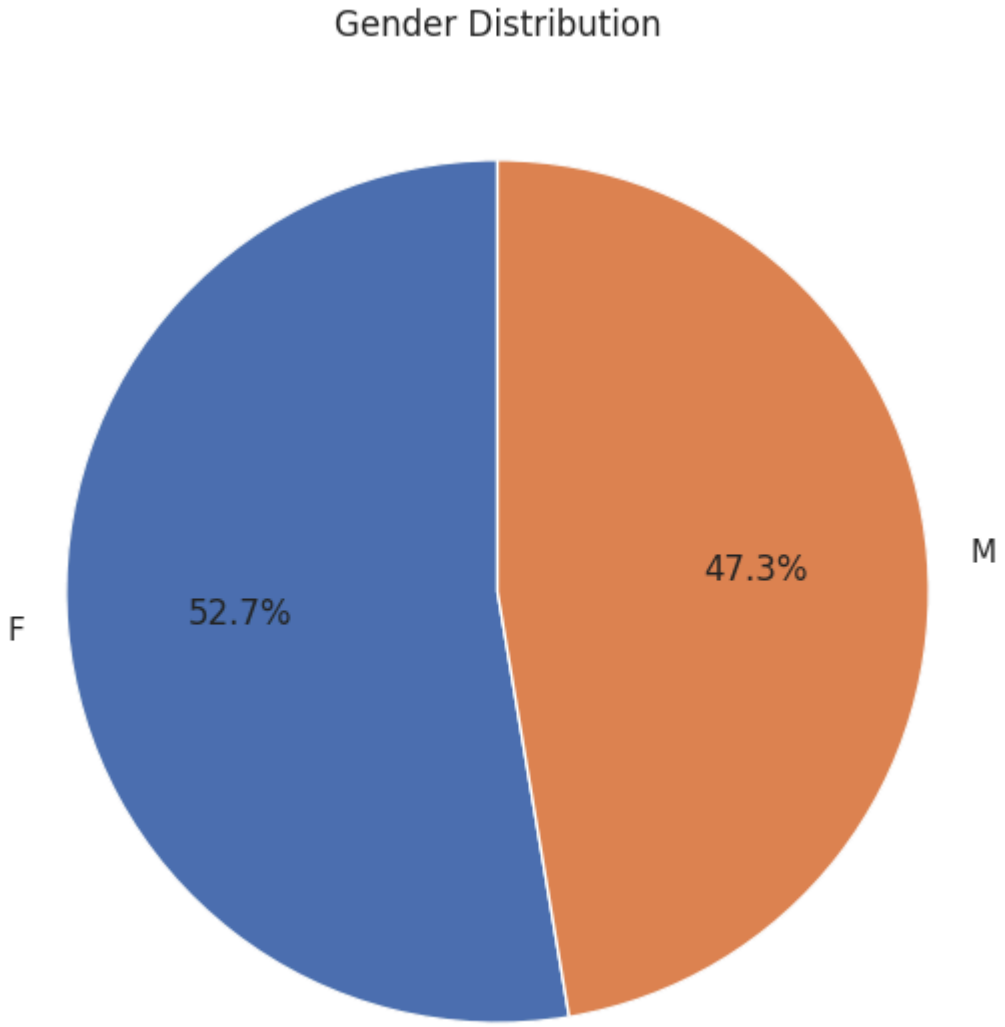
16. Pie Chart — Gender Distribution

In [36]:

```
gender_counts = df["sex"].value_counts()

plt.figure(figsize=(7, 7))
plt.pie(gender_counts.values, labels=gender_counts.index, autopct="%1.1f%%", startar
plt.title("Gender Distribution")
plt.show()
```

Out[36]:

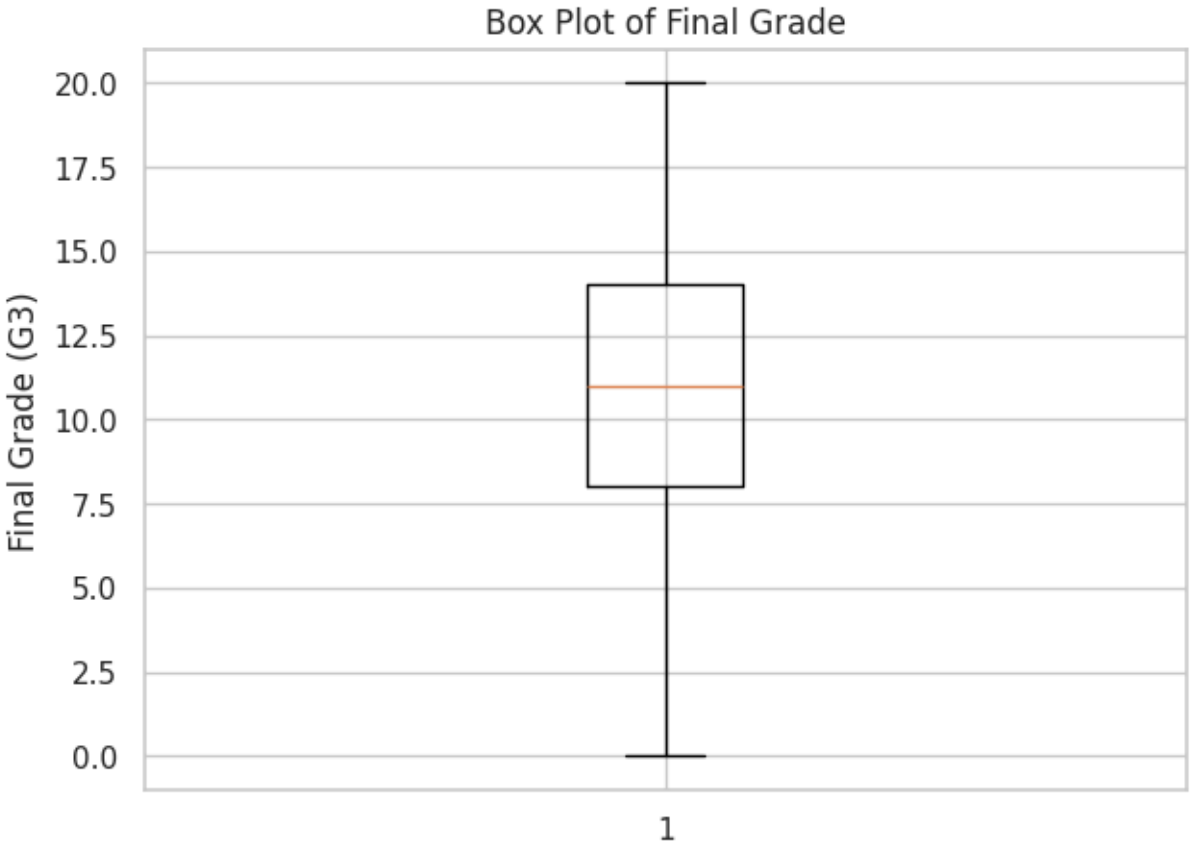


> **Observation:** The data is almost evenly split: 208 girls (about 53%) and 187 boys (about 47%).

17. Box Plot — Final Grade

In [37]:
```
plt.figure(figsize=(7, 5))
plt.boxplot(df["G3"])
plt.ylabel("Final Grade (G3)")
plt.title("Box Plot of Final Grade")
plt.show()
```

Out[37]:



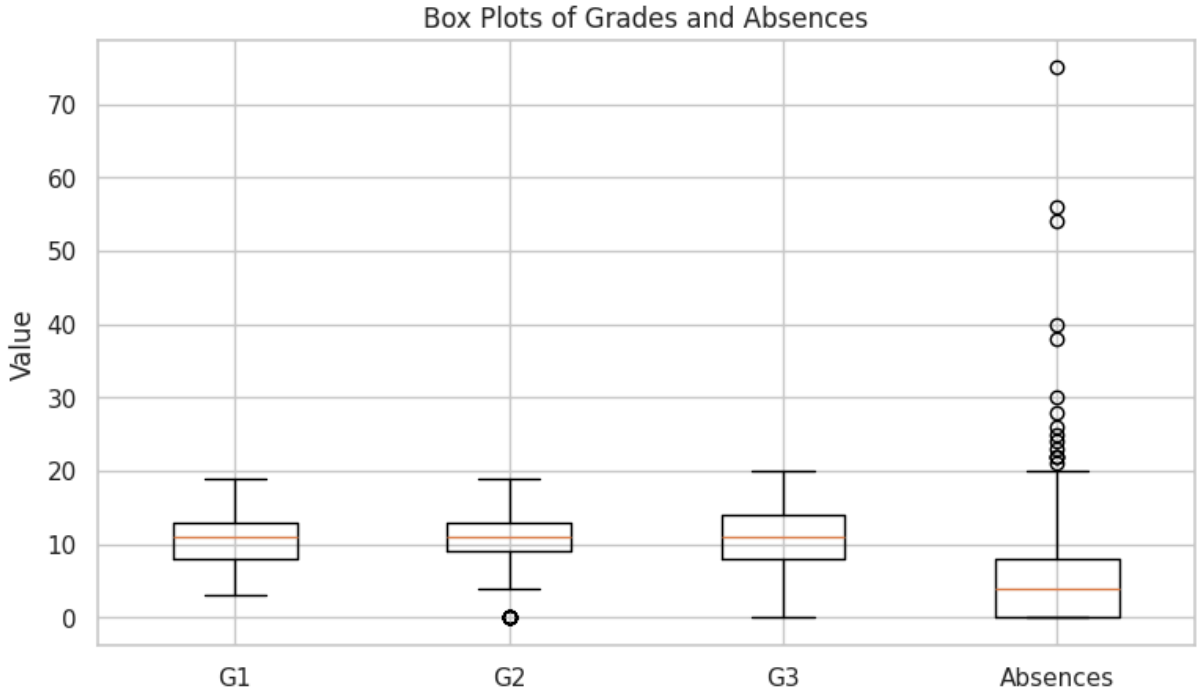
> **Observation:** The middle half of students score between 8 and 14. The middle line (median) is at 11, and scores spread from 0 to 20.

18. Multiple Box Plots — Grades and Absences

In [38]:

```
plt.figure(figsize=(9, 5))
plt.boxplot(
    [df["G1"], df["G2"], df["G3"], df["absences"]],
    labels=["G1", "G2", "G3", "Absences"]
)
plt.title("Box Plots of Grades and Absences")
plt.ylabel("Value")
plt.show()
```

Out[38]:



> **Observation:** G1, G2 and G3 look alike, with the median around 11 each. G2 shows a few low dots (students scoring very low). Absences are much lower for most students but have many high dots, meaning a small group misses a lot of classes.

19. Subplots

In [39]:

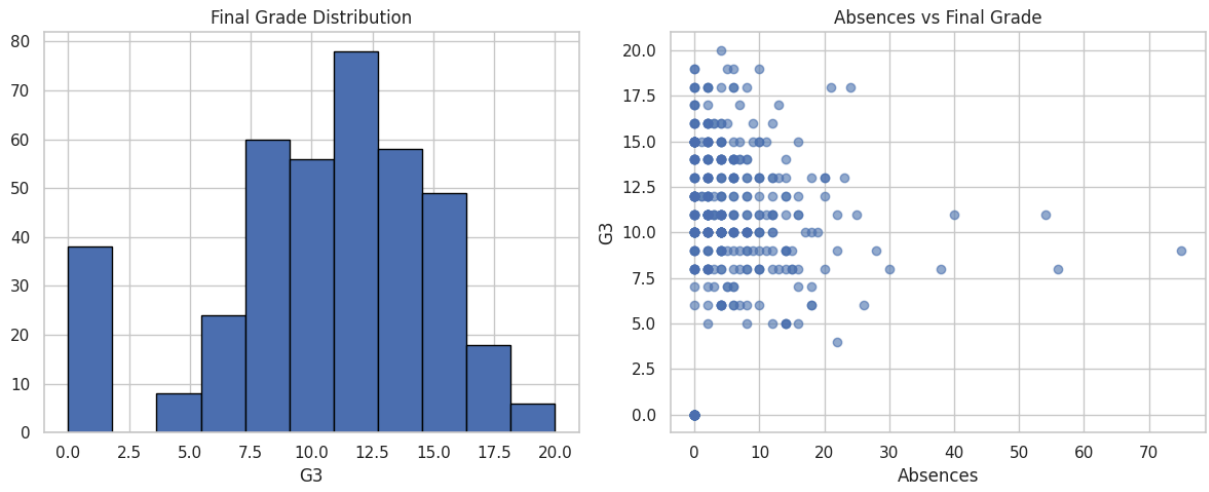
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].hist(df["G3"], bins=11, edgecolor="black")
axes[0].set_title("Final Grade Distribution")
axes[0].set_xlabel("G3")

axes[1].scatter(df["absences"], df["G3"], alpha=0.6)
axes[1].set_title("Absences vs Final Grade")
axes[1].set_xlabel("Absences")
axes[1].set_ylabel("G3")

plt.tight_layout()
plt.show()
```

Out[39]:



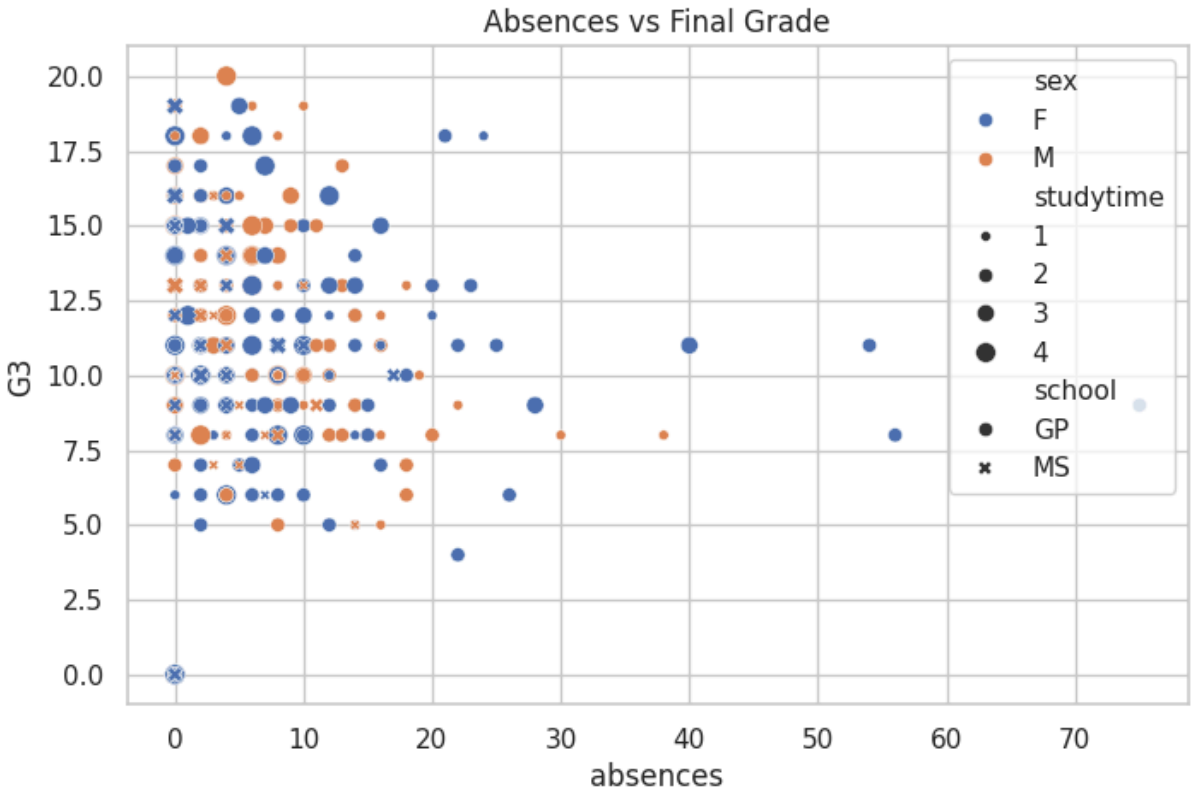
> **Observation:** The left chart is the grade histogram and the right chart is the absences scatter plot. Together they show that most grades fall in the middle and absences do not strongly affect grades.

Seaborn Visualizations

20. Seaborn Scatter Plot

In [40]:
```
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x="absences",
    y="G3",
    hue="sex",
    style="school",
    size="studytime"
)
plt.title("Absences vs Final Grade")
plt.show()
```

Out[40]:



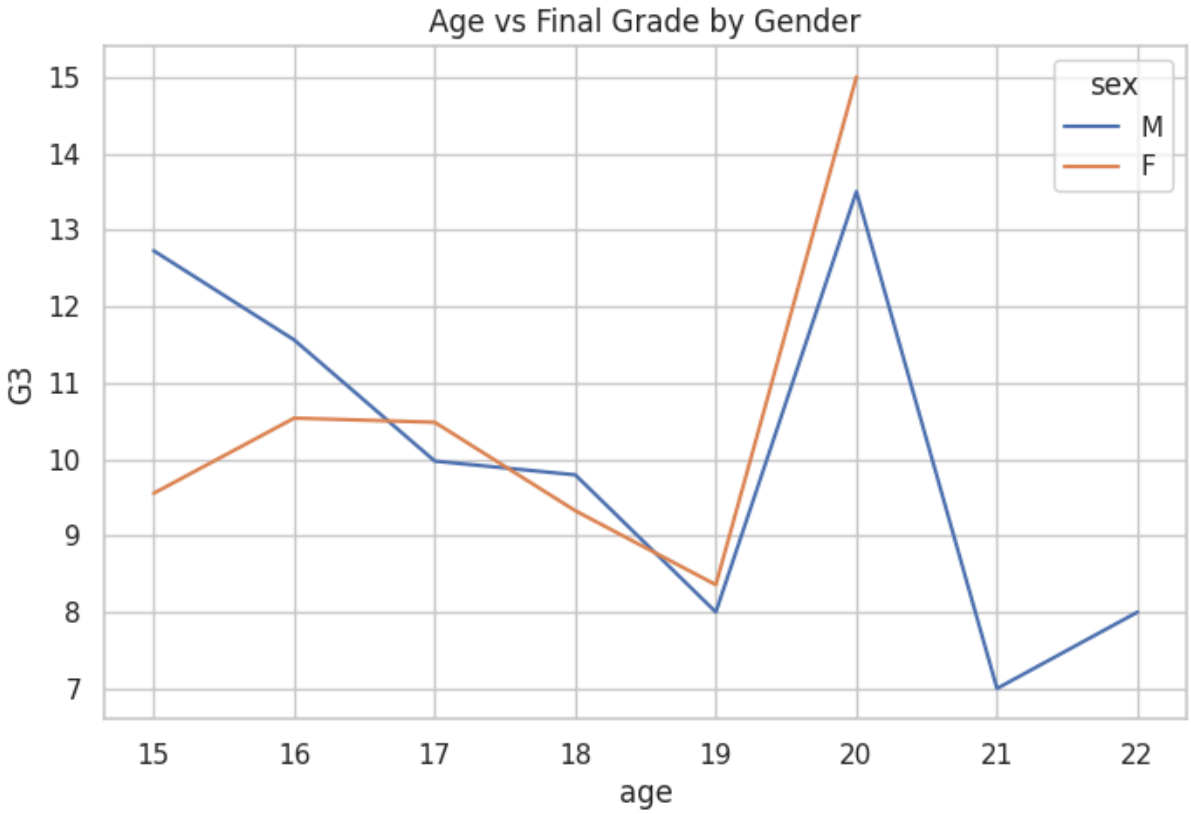
> **Observation:** This is the same absences vs grade chart, now with colour for gender, shape for school and size for study time. Dots are mixed everywhere, so none of these details shows a clear pattern with absences and grades.

21. Seaborn Line Plot

In [41]:
```
plt.figure(figsize=(8, 5))
sns.lineplot(
    data=df.sort_values("age"),
```

```
    x="age",
    y="G3",
    hue="sex",
    errorbar=None
)
plt.title("Age vs Final Grade by Gender")
plt.show()
```

Out[41]:



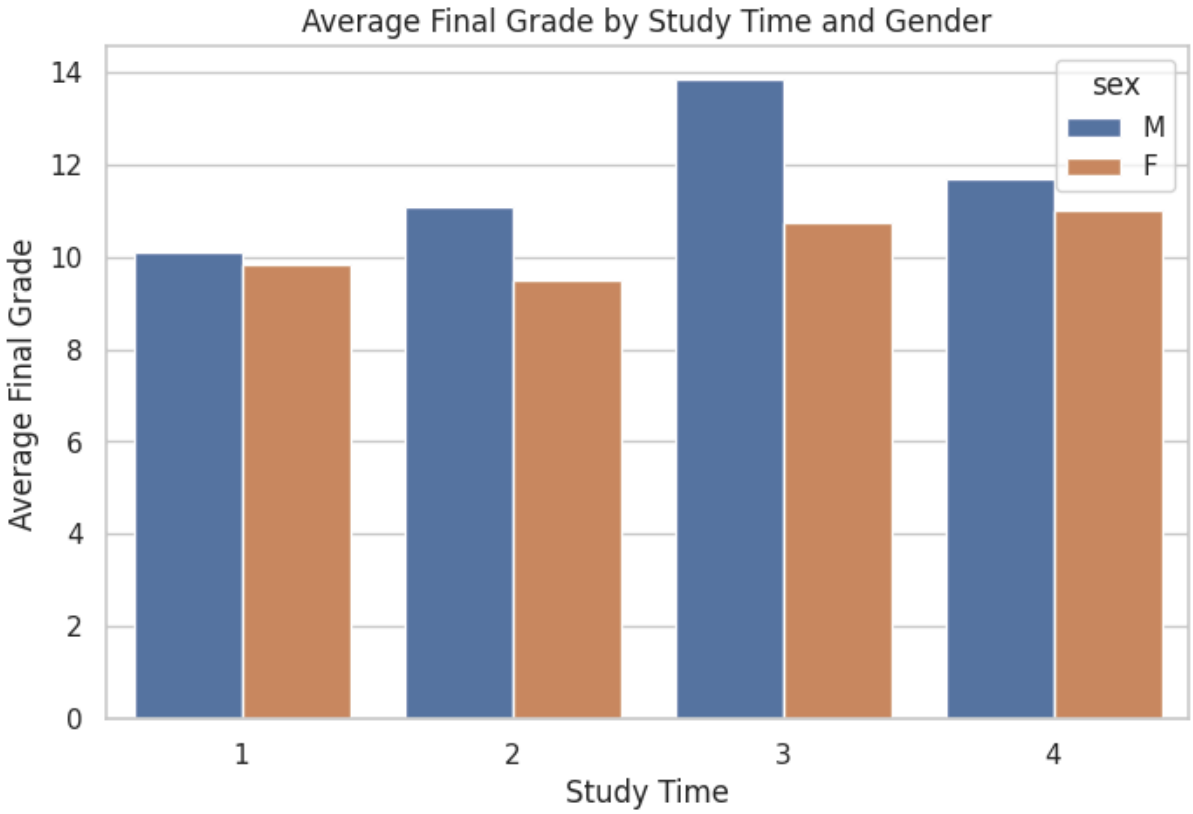
> **Observation:** The average grade goes down as students get older: about 11.3 at age 15 and about 8.2 at age 19. The lines jump around after age 19 because there are only a few students that old.

22. Seaborn Bar Plot

In [42]:

```
plt.figure(figsize=(8, 5))
sns.barplot(data=df, x="studytime", y="G3", hue="sex", errorbar=None)
plt.xlabel("Study Time")
plt.ylabel("Average Final Grade")
plt.title("Average Final Grade by Study Time and Gender")
plt.show()
```

Out[42]:



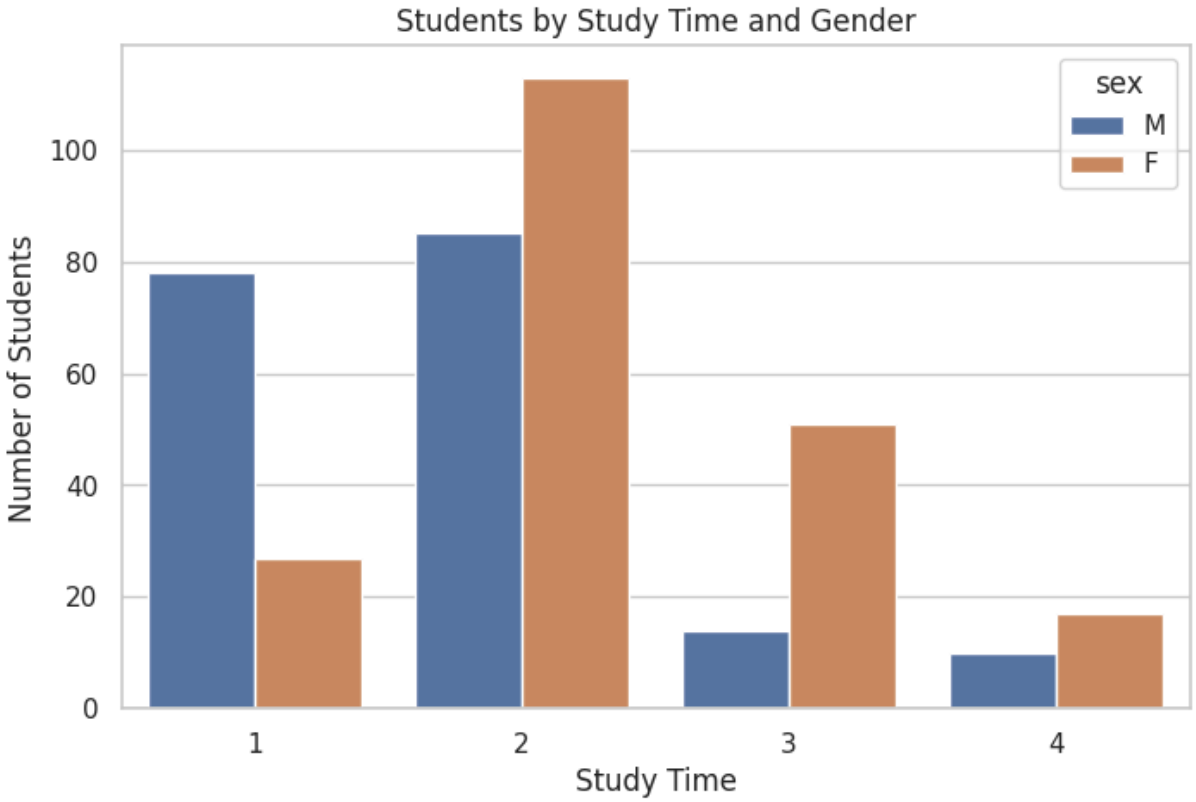
> **Observation:** This shows the same result as the earlier grouped bar chart. Grades are generally higher for students who study more, and boys at study level 3 score the highest.

23. Seaborn Count Plot

In [43]:

```
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="studytime", hue="sex")
plt.xlabel("Study Time")
plt.ylabel("Number of Students")
plt.title("Students by Study Time and Gender")
plt.show()
```

Out[43]:



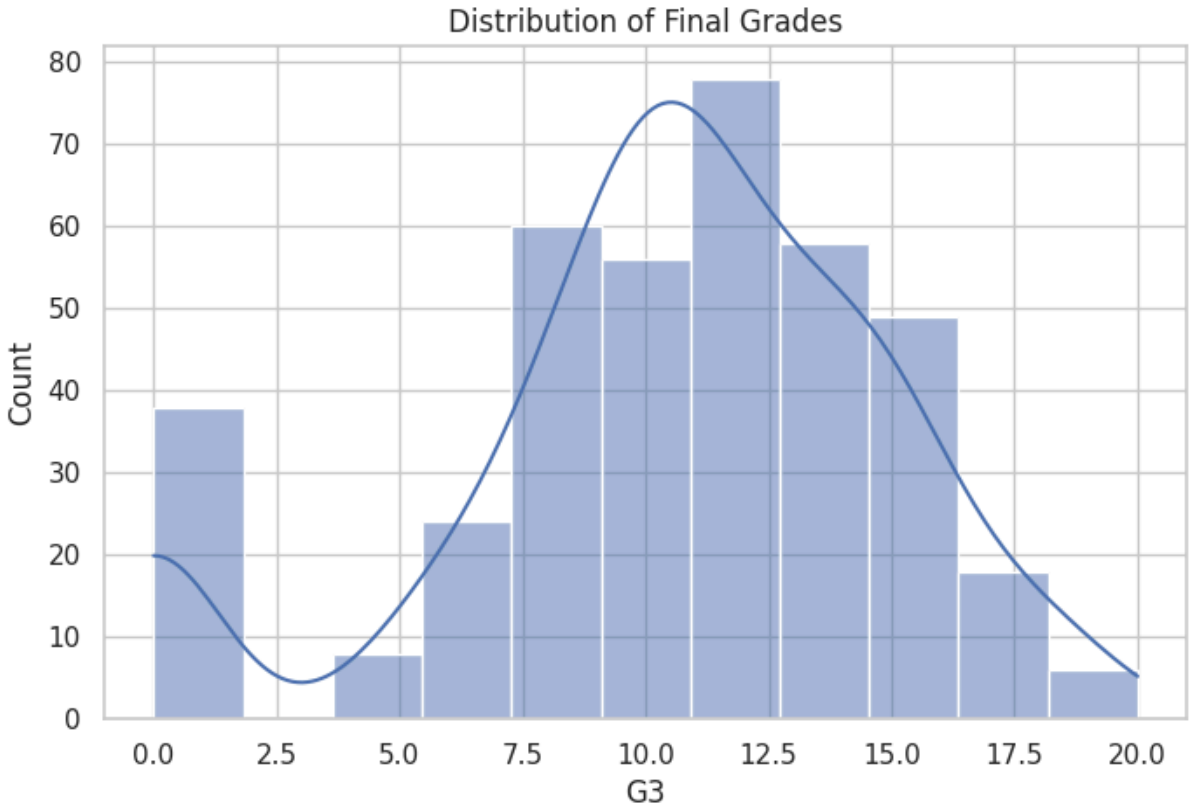
> **Observation:** Most students study at level 2 (198 students), then level 1 (105), then level 3 (65), and very few at level 4 (27). Boys are more common at level 1, and girls are more common at levels 2, 3 and 4.

24. Seaborn Histogram

In [44]:

```
plt.figure(figsize=(8, 5))
sns.histplot(data=df, x="G3", bins=11, kde=True)
plt.title("Distribution of Final Grades")
plt.show()
```

Out[44]:



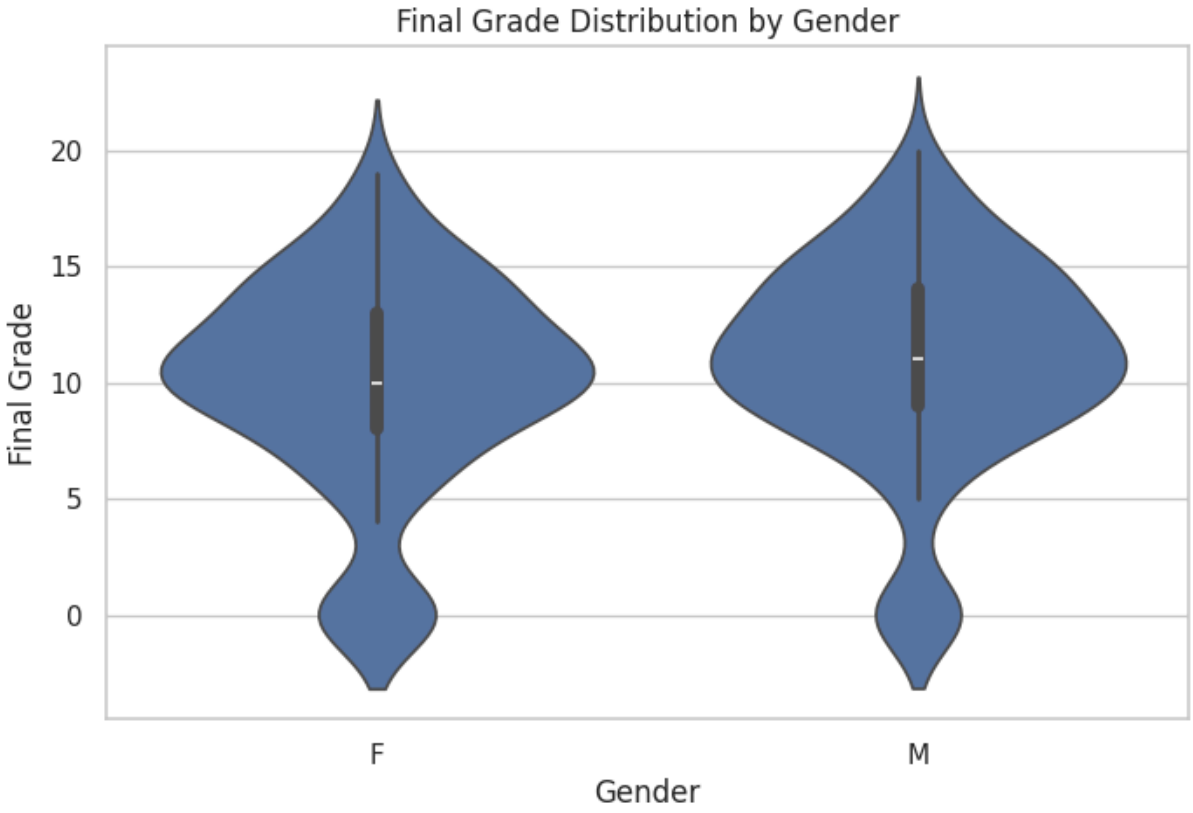
> **Observation:** The grades pile up around 10 and 11, with a long tail to the left because of low scores and zeros. The curve shows the shape of the grades smoothly.

25. Seaborn Violin Plot

In [45]:

```
plt.figure(figsize=(8, 5))
sns.violinplot(data=df, x="sex", y="G3")
plt.xlabel("Gender")
plt.ylabel("Final Grade")
plt.title("Final Grade Distribution by Gender")
plt.show()
```

Out[45]:



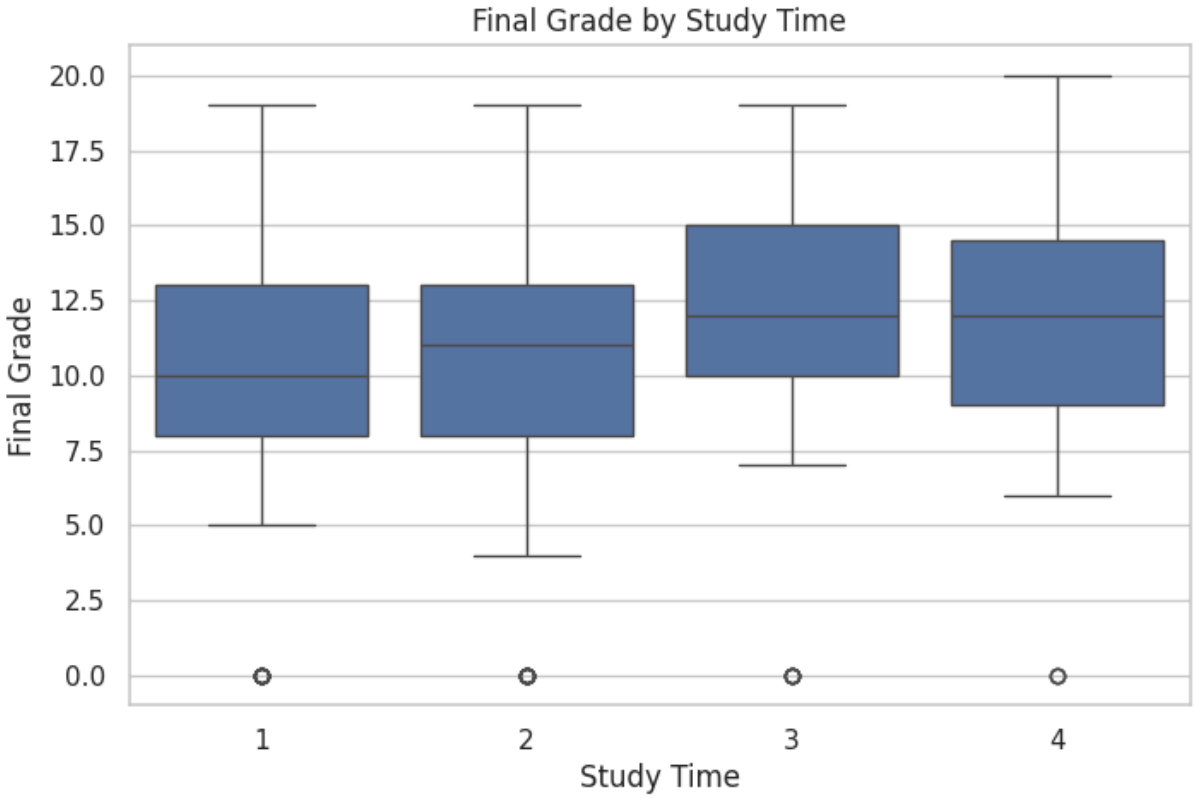
> **Observation:** Both shapes are widest between 10 and 12. Boys' middle value is a little higher (11 vs 10). Both have thin tails down to 0.

26. Seaborn Box Plot

In [46]:

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="studytime", y="G3")
plt.xlabel("Study Time")
plt.ylabel("Final Grade")
plt.title("Final Grade by Study Time")
plt.show()
```

Out[46]:



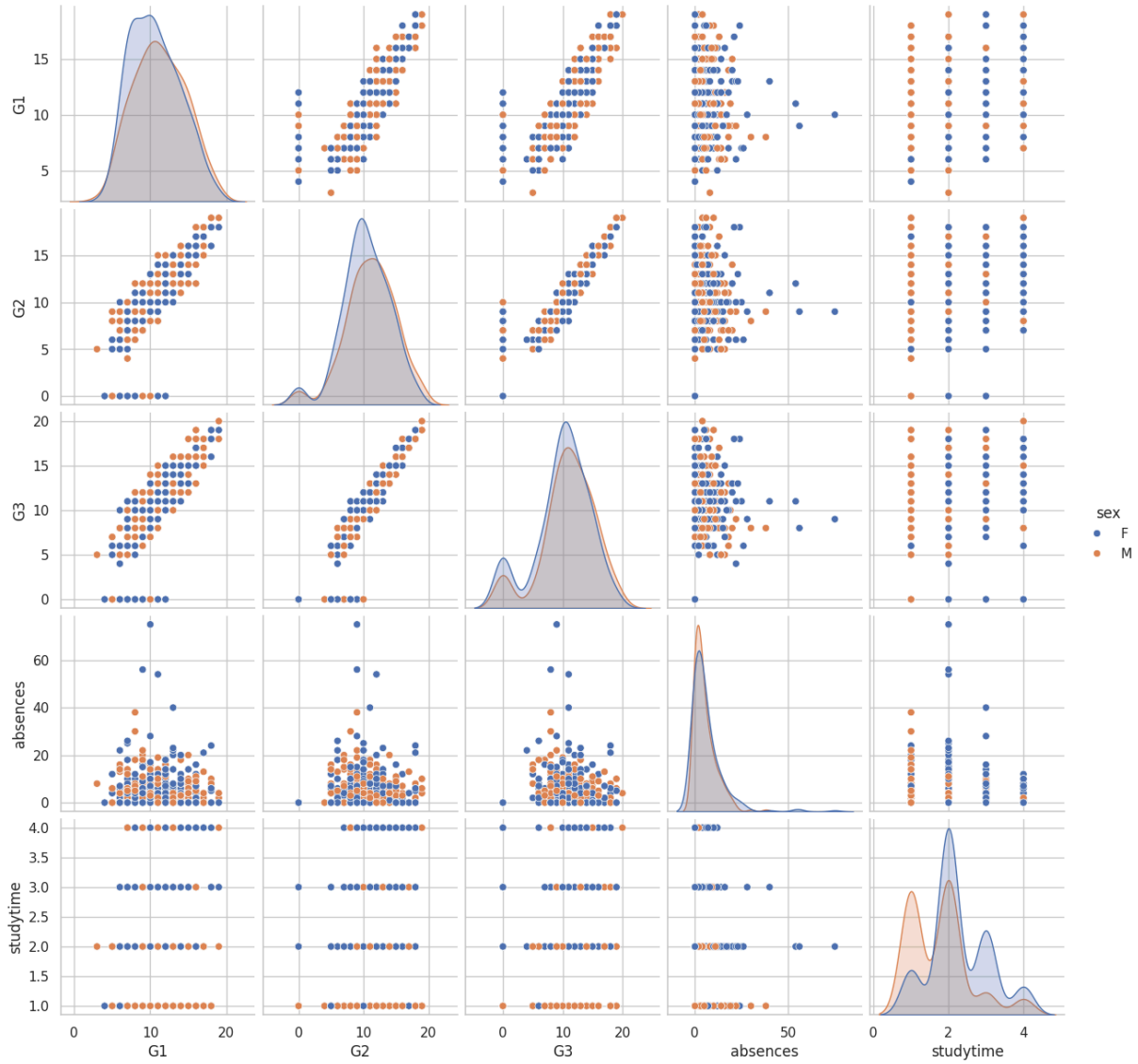
> **Observation:** The middle grade goes up with study time: about 10 at level 1, 11 at level 2, and 12 at levels 3 and 4. Students who study more tend to score higher.

27. Pair Plot

In [47]:

```
sns.pairplot(
    df[["G1", "G2", "G3", "absences", "studytime", "sex"]],
    hue="sex"
)
plt.show()
```

Out[47]:



> **Observation:** G1, G2 and G3 rise together almost in a straight line, so a student who does well in the first test usually does well later. Absences and study time show no clear pattern with grades. Boys and girls overlap a lot.

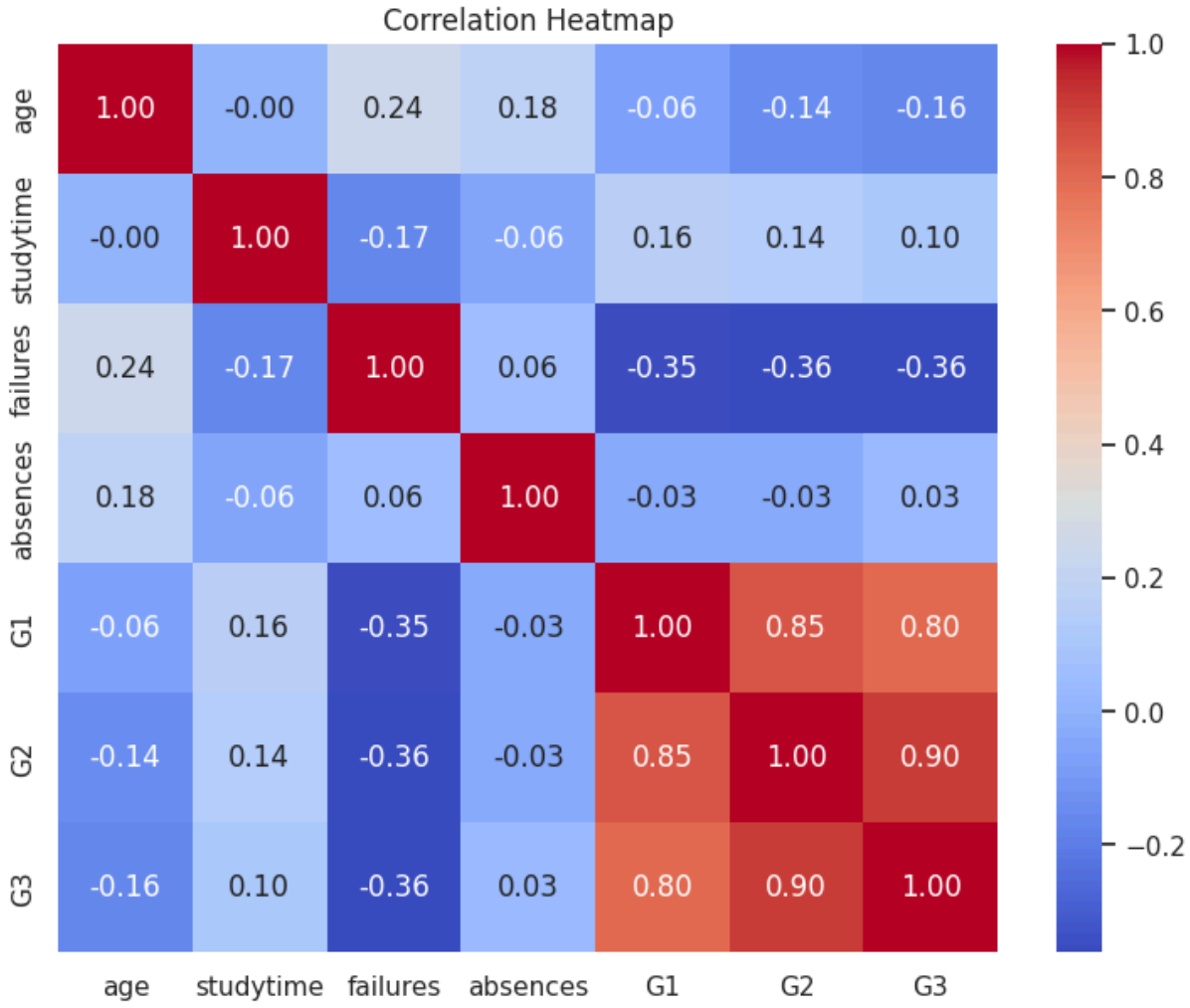
28. Correlation Heatmap

In [48]:

```
correlation = df[[
    "age", "studytime", "failures", "absences", "G1", "G2", "G3"
]].corr()

plt.figure(figsize=(9, 7))
sns.heatmap(correlation, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Out[48]:



> **Observation:** G2 and G3 are very strongly linked (0.90), and G1 and G3 are also strong (0.80). Failures have a mild negative link with grades (about -0.36), meaning more past failures go with lower grades. Age has a small negative link (-0.16), study time a small positive link (0.10), and absences almost none (0.03).

Conclusion

The Student Performance dataset was analyzed using **Pandas**, **Matplotlib**, and **Seaborn**.

The notebook demonstrates: - Data loading and inspection - Selection and filtering - Creating calculated columns - Missing-value checking and cleaning - Grouping and aggregation - Sorting - Pivot tables - Data export - Line, bar, grouped bar, scatter, histogram, pie and box plots - Subplots - Seaborn scatter, line, bar, count, histogram, violin and box plots - Pair plot and correlation heatmap

These operations follow the same major concepts demonstrated in the supplied class material, adapted to the Student Performance dataset.